Machine Translated by Google



UNIVERSIDAD DE VALENCIA • ETSE-UV

2.2 Organización de Registros en un Archivo y Técnicas de Indexación

Sistemas de Gestión de Bases de Datos (SGBD) • Tema 2.b

Profesor: Javier Sevilla — Departamento de Informática

Grado en Ingeniería Telemática y Grado en Ingeniería Informática

Machine Translated by Google

Tema 2.b: Contenidos de la Unidad

1. Organización de Archivos

- Archivos secuenciales no ordenados: Munts (heap files).
- Archivos secuenciales ordenados: Búsqueda binaria y desbordamiento.
- Ordenación externa: Fusión recursiva (External Merge Sort).
- Archivos dispersos: Hashing estático, cubetas y colisiones.

2. Técnicas de Indexación

- Concepto y tipos de índices: Primario, secundario, denso y disperso.
- Índices de agrupamiento: Clustering vs non-clustering.
- Índices multinivel: Árboles B+ de orden n.
- Evaluación y análisis de costes de E/S: Modelos matemáticos.

Machine Translated by Google

Organización de los Registros en Archivos

La elección de la organización de un archivo depende estrictamente del uso previsto y del compromiso entre el tiempo de acceso (lectura) y el tiempo de modificación (inserción y borrado).

SECUENCIAL NO ORDENADO

Heap o Munt

Acumula registros a medida que llegan. Inserción inmediata al final o agujeros libres; lectura costosa.

SECUENCIAL ORDENADO

Archivo Ordenado

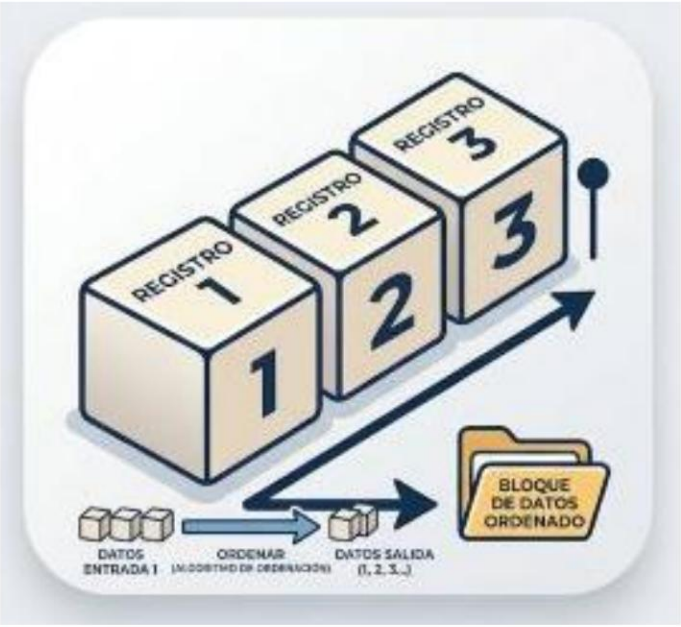
Registros ordenados según una clave. Lectura secuencial y por rango rápida; inserciones mucho costosas.

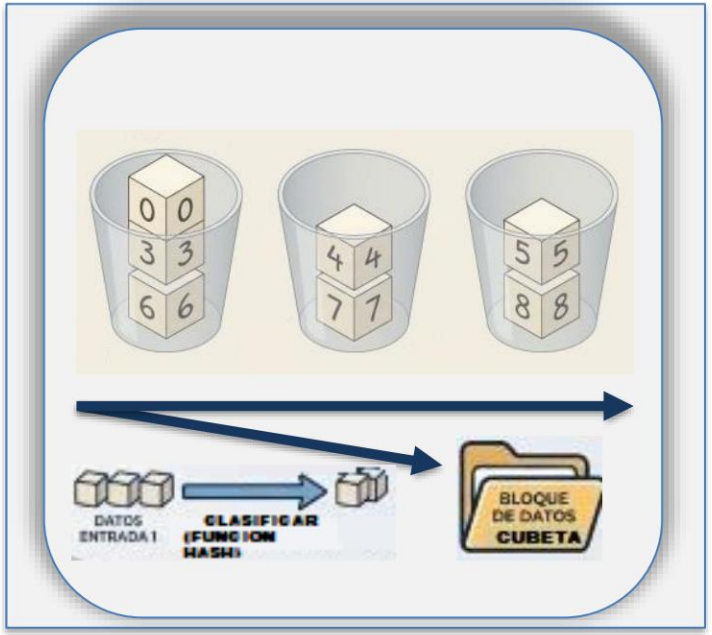
DISPEROS (HASHING)

Archivo Hash

Una función matemática calcula directamente la dirección de la cubeta de disco. Acceso casi directo por clave.







Machine Translated by Google

Archivos Secuenciales No Ordenados (Heap Files)



Características Fundamentales

- Es la organización más básica: consiste en acumular registros.
- No es necesario ningún método elaborado para determinar la posición de un registro nuevo.
- Inserción muy rápida: Se escribe directamente al final del archivo.
- Muy eficiente cuando el acceso típico es el recorrido completo de toda su relación.

El Reto del Borrado

Si sólo se insertaran tuplas nuevas, bastaría con añadirlas continuamente al final del archivo a medida que se generan.

¿Qué hacemos cuando se debe eliminar un registro existente?

Evitar dejar agujeros vacíos inutilizados sin degradar gravemente la tasa de transferencia de E/S.

Machine Translated by Google

Gestión del Borrado en Archivos Secuenciales No Ordenados (Heap Files)



1. Compactación

Al eliminar un registro, todos los registros posteriores desplazan una posición hacia arriba.

Desventaja: Coste de E/S altísimo para reescribir múltiples blogs.

2. Reemplazo

El agujero liberado se ocupa copiando el último registro del archivo físico.

Ventaja: Sólo requiere escribir un único blog y truncar el final.

3. Marcaje Lógico

El registro se marca como borrado mediante un bit o puntero de encadenamiento.

Ventaja: Se mantiene una lista libre para futuras inserciones.

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Machine Translated by Google

Estructura de Lista Libre en Archivos Secuenciales No Ordenados (Heap Files)



Mecanismo de la Lista Enlazada

La cabecera del archivo (header) almacena la dirección del primer registro vacío. Cada posición eliminada actúa como un puntero hacia el siguiente agujero libre.

Procedimiento de Inserción

1. La nueva tupla se instala en la primera posición apuntada por la cabecera, (riego 1).
2. Actualizar el puntero de inicio con el siguiente de la cadena (riego 4).
   Si la lista está vacía, se escribe al final.

Posición	ID	Nombre	Departamento	Salario
header	Puntero en el riego 1 (Lista libre)			
riego 0	10101	Srinivasan	Comp. Sci.	65000
riego 1	[BORRADO] → Puntero a riego 4			
riego 2	15151	Mozart	Music	40000
riego 3	22222	Einstein	Physics	95000
riego 4	[BORRADO] → Puntero a riego 6			
riego 5	33456	Gold	Physics	87000
riego 6	[BORRADO] → Fin de lista (null)			

Generalmente, el header está en memoria,
Por lo que E/S sobre header no se contabiliza

Machine Translated by Google

Operaciones Básicas en Archivos Secuenciales No Ordenados (Heap Files)



Búsqueda

Exige una exploración secuencial recorriendo blog a blog desde el inicio.

COSTE DE E/S (PEOR CASO)
Bx blogs

Inserción

Se añade al final o se aprovecha el primer agujero de la lista libre.

COSTE DE E/S TÍPICO
2 bloques (I+W)

Borrado

Requiere localizar el registro, marcarlo y reescribir el blog a disco.

COSTE DE E/S TOTAL
Búsqueda + 1 W

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Machine Translated by Google

El Reto de la Ordenación de Grandes Volúmenes



¿Por qué ordenar los archivos?

- La búsqueda se hace mucho más eficiente mediante búsqueda binaria o índices.
- Las consultas por rango de valores (ej. salarios entre 30K y 50K) se ejecutan rápidamente.
- Muchos operadores relacionales como JOIN (merge-join) exigen archivos previamente ordenados.

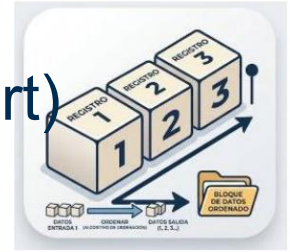
Limitación de la Memoria Principal (RAM)

Los algoritmos convencionales (Quicksort, Heapsort) asumen que todos los datos caben en RAM.

En un SGBD, ¡la mesa es mucho mayor que la memoria disponible!
Es necesario aplicar técnicas apoyadas en memoria secundaria: Ordenación Externa (External Merge Sort).

Machine Translated by Google

Algoritmo de Ordenación Externa (Merge Sort)



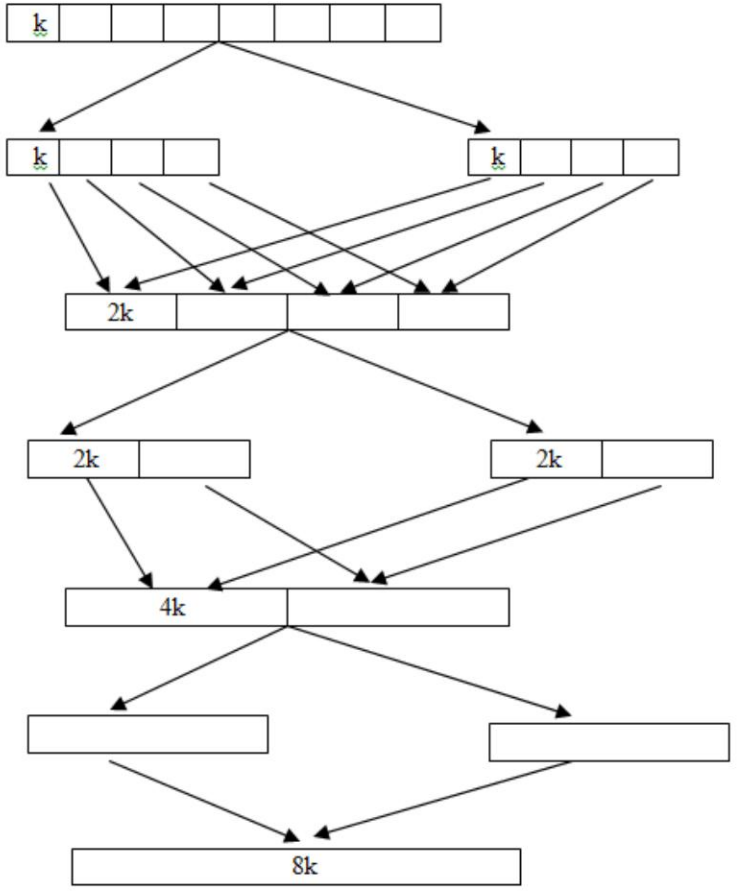
Estrategia en dos fases: (1) Generación de tramos iniciales ordenados en RAM y (2) Fusiones ordenadas recursivas (merge pasos).

Fase 1: Tramos Iniciales (Runs)

Se divide el archivo en trozos de tamaño k (donde k es el número de marcos de memoria RAM disponibles). Cada tramo se carga a memoria, se ordena internamente y se vuelca a disco.

Fase 2: Fusión Recursiva

Utilizando archivos auxiliares (F1 y F2), se fusionan pares de tramos ordenados generando bloques de tamaño creciente (2k, 4k, 8k...) hasta alcanzar el tamaño total de la tabla.



Machine Translated by Google

Archivos Secuenciales Ordenados: Balance



Ventajas Clave

- Recorrido ordenado: Lectura secuencial directa sin ordenación previa.
- Búsqueda eficiente: Permite aplicar búsqueda binaria sobre el campo de ordenación.
- Consultas por rango: Localizada la primera tupla, el resto se leen correlativamente.

Inconvenientes Estructurales

- Buscar por otros campos: Tan ineficiente como un archivo desordenado (O(N)).
- Inserciones y borrados críticos: Es necesario desplazar los registros para mantener el orden físico.
- La modificación de la clave de ordenación implica un borrado seguido de una inserción.

Machine Translated by Google

Gestión de Inserciones en Archivos Ordenados



Método 1: Desplazamiento Físico

Se desplazan físicamente todos los registros posteriores a la posición en la que debe entrar la nueva tupla.

Coste: Desmedido en tablas grandes; puede implicar reescribir miles de blogs a disco.

Método 2: Archivo de Desbordamiento

La nueva tupla se ubica en un archivo de overflow y se enlaza mediante un puntero desde el registro anterior al área principal.

Mantenimiento: Exige reorganizaciones periódicas del archivo para eliminar los encadenamientos.

Archivo Datos

ID	INFO	SEG
1	ANA	
2	ABEL	
3	CARLOS	
4	SARA	

Archivo Overflow

ID	INFO	SEG

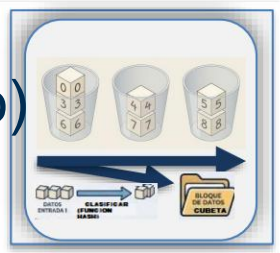
Insertar “JAVIER” …..

ID	INFO	SEG
1	ANA	
2	ABEL	
3	CARLOS	1
4	SARA	

ID	INFO	SEG
1	JAVIER	4

Machine Translated by Google

Archivos Dispersos (Hashing Estático)



Concepto: Permite un acceso directo (sin exploración) mediante una función matemática f(k) aplicada sobre un campo de racimo (normalmente la clave primaria).

CUBETA (BUCKET)

Unidad de Almacenamiento

Blog o conjunto de bloques de disco contiguos que contienen los registros que comparten un mismo código hash.

FUNCIÓN HASH

f(k) = d

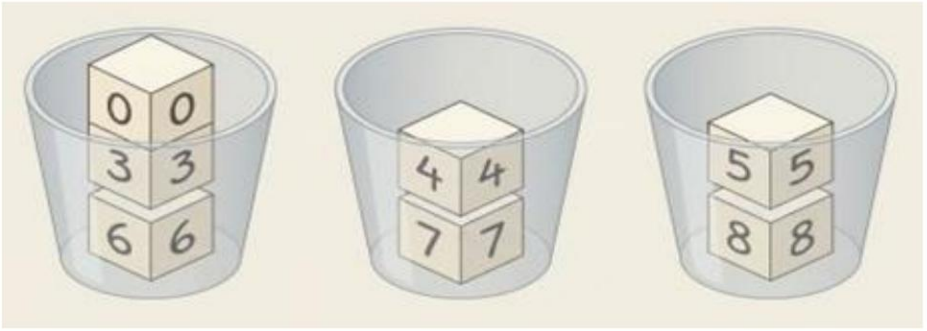
Mapea el valor de la clave k en un entero d que corresponde a la dirección física de la cubeta.

ACCESO O(1)

Rendimiento Directo

Permite recuperar un registro en un único acceso a disco en ausencia de colisiones.

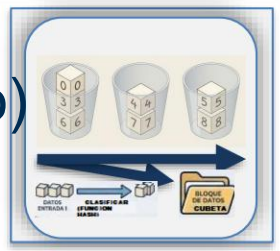
Archivo Datos FUNCIÓN HASH f(k) = valor mod 3



Cubeta 0 Cubeta 1 Cubeta 2

Machine Translated by Google

Archivos Dispersos (Hashing Estático)



Concepto: Permite un acceso directo (sin exploración) mediante una función matemática f(k) aplicada sobre un campo de racimo (normalmente la clave primaria).

CUBETA (BUCKET)

Unidad de Almacenamiento

Blog o conjunto de bloques de disco contiguos que contienen los registros que comparten un mismo código hash.

FUNCIÓN HASH

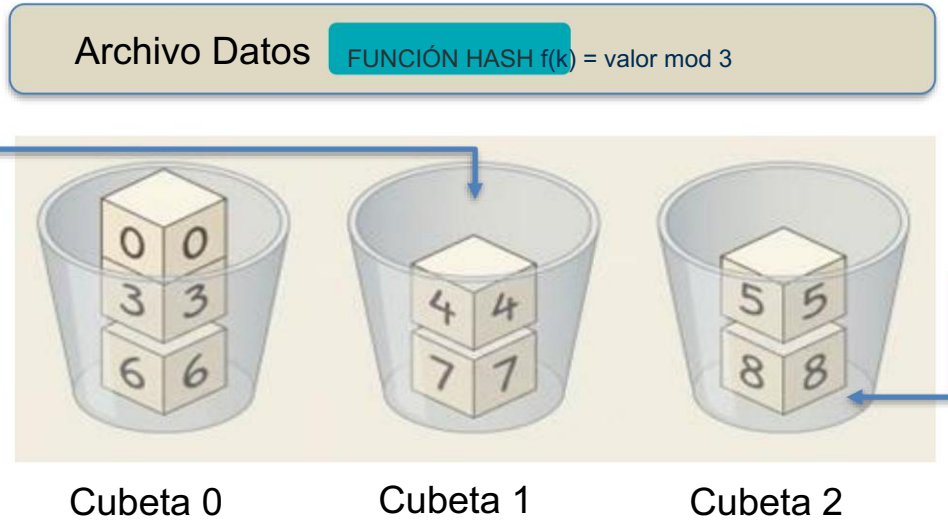
f(k) = d

Mapea el valor de la clave k en un entero d que corresponde a la dirección física de la cubeta.

ACCESO O(1)

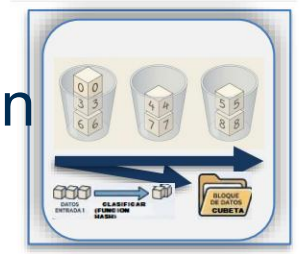
Rendimiento Directo

Permite recuperar un registro en un único acceso a disco en ausencia de colisiones.



Machine Translated by Google

Diseño de Funciones de Dispersión



Requisitos de una Buena Función Hash

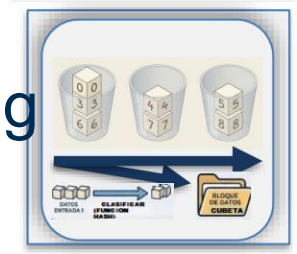
- Fácil y rápida de calcular: No debe sobrecargar la CPU.
- Distribución uniforme: Asigna tuplas uniformemente entre las N cubetas.
- Minimización de colisiones: Evita puntos calientes o cubetas saturadas.

Técnicas Clásicas de Cálculo

- División y residuo (Módulo): (con N preferentemente primero). $f(k) = k \bmod N$
- Truncamiento: Extraer dígitos específicos de la clave.
- Pliegue: Sumar segmentos de la clave (ej. 1234 + 5678).
- Multiplicación: Extraer los bits centrales del cuadrado de la llave.

Machine Translated by Google

El Problema de las Colisiones en Hashing



Se produce una colisión cuando dos claves distintas generan el mismo código hash (f(k1) = f(k2)). Si la cubeta se llena, gestionar el desbordamiento (overflow).

Direccionamiento Cerrado

Se mantiene una lista enlazada de cubetas de desbordamiento para cada cubeta principal saturada.

MÁS USADO EN SGBD

Direccionamiento Abierto

Si la cubeta está llena, se busca secuencialmente la primera posición libre en cubetas contiguas.

DEGRADA EL RENDIMIENTO

Dispersión Múltiple

Se aplica una segunda función hash f2(k) si la primera colisiona antes de recorrer otras posiciones.

DOBLE HASH

Machine Translated by Google

Direccionamiento Cerrado con Cubetas de Desbordamiento



Estructura de Encadenamiento

Cada cubeta principal dispone de un puntero hacia una lista de bloques auxiliares. Las consultas acceden directamente a la cubeta base y, sólo si es necesario, recorren los bloques de overflow.

Impacto en el Coste de E/S

En el mejor caso, el coste es de 1 solo blog. Si existen cadenas largas de desbordamiento, el tiempo de acceso degenera hacia una búsqueda lineal.

Cubeta 0	Registros [0]	
Cubeta 1	Lleno →	Overflow 1A → Overflow 1B
Cubeta 2	Registros [2]	
Cubeta 3	Registros [3]	

Machine Translated by Google

Parámetros para la Evaluación de Costes de E/S

Métricas de Archivo y Mesa

- TB : Tamaño del blog de la base de datos (bytes).
- Tx : Número total de registros de la tabla.
- Lx : Longitud en bytes de cada registro de la mesa.
- FBx : Factor de bloque de la tabla (Registros por blog) = TB / Lx
- Bx : Número total de bloques de la tabla = Tx / FBx
- BPCx : Bloques por cubeta en organización dispersa.

DISCUSIÓN: ¿Cuánto cuesta encontrar por un campo y leer un registro de un archivo?

1. Si el archivo está ordenado
2. Si el archivo está ordenado por ese campo
3. Si el archivo tiene una organización dispersa y la fórmula utiliza ese campo
4. Si el archivo está ordenado por otro campo, o el campo no está en la fórmula

Machine Translated by Google

Parámetros para la Evaluación de Costes de E/S

Métricas de Archivo y Mesa

- **TB** : Tamaño del blog de la base de datos (bytes).
- **Tx** : Número total de registros de la tabla.
- **Lx** : Longitud en bytes de cada registro de la mesa.
- **FBx** : Factor de bloque de la tabla (Registros por blog) = TB / Lx
- **Bx** : Número total de bloques de la tabla = Tx / FBx
- **BPCx** : Bloques por cubeta en organización dispersa.

DISCUSIÓN: ¿Cuánto cuesta encontrar por un campo y leer un registro de un archivo?

1. Si el archivo está ordenado



2. Si el archivo está ordenado por ese campo
3. Si el archivo tiene una organización dispersa y la fórmula utiliza ese campo
4. Si el archivo está ordenado por otro campo, o el campo no está en la fórmula

Fichero **NO** Ordenado

	Apellido	Edad	Ciudad
Bloque 1	Aliaga	23	Bilbao
	Cardenas	18	Bilbao
	Gimenez	21	Barcelona
	Martínez	22	Madrid
Bloque 2	Cerdán	22	Bilbao
	García	22	Valencia
	Yañez	21	Barcelona
	Gomez	23	Barcelona
Bloque 3	Lopez	20	Valencia
	Tarraga	21	Barcelona
	Nuñez	20	Madrid
	Romero	24	Valencia
Bloque 4	Alcocer	25	Barcelona
	Vilchez	23	Madrid
	Aldomar	22	Valencia



Coste =

Machine Translated by Google

Parámetros para la Evaluación de Costes de E/S

Métricas de Archivo y Mesa

- TB : Tamaño del blog de la base de datos (bytes).
- Tx : Número total de registros de la tabla.
- Lx : Longitud en bytes de cada registro de la mesa.
- FBx : Factor de bloque de la tabla (Registros por blog) = TB / Lx
- Bx : Número total de bloques de la tabla = Tx / FBx
- BPCx : Bloques por cubeta en organización dispersa.

DISCUSIÓN: ¿Cuánto cuesta encontrar por un campo y leer un registro de un archivo?

1. Si el archivo está ordenado

2. Si el archivo está ordenado por ese campo



3. Si el archivo tiene una organización dispersa y la fórmula utiliza ese campo

4. Si el archivo está ordenado por otro campo, o el campo no está en la fórmula

Archivo ordenado por Apellido



	Apellido	Edad	Ciudad
Bloque 1	Alcocer	25	Barcelona
	Aldomar	22	Valencia
	Aliaga	23	Bilbao
	Cardenas	18	Bilbao
Bloque 2	Cerdán	22	Bilbao
	García	22	Valencia
	Gimenez	21	Barcelona
	Gomez	23	Barcelona
Bloque 3	Lopez	20	Valencia
	Martínez	22	Madrid
	Nuñez	20	Madrid
	Romero	24	Valencia
Bloque 4	Tarraga	21	Barcelona
	Vilchez	23	Madrid
	Yañez	21	Barcelona

Coste =

Machine Translated by Google

Parámetros para la Evaluación de Costes de E/S

Métricas de Archivo y Mesa

- TB : Tamaño del blog de la base de datos (bytes).
- Tx : Número total de registros de la tabla.
- Lx : Longitud en bytes de cada registro de la mesa.
- FBx : Factor de bloque de la tabla (Registros por blog) = TB / Lx
- Bx : Número total de bloques de la tabla = Tx / FBx
- BPCx : Bloques por cubeta en organización dispersa.

DISCUSIÓN: ¿Cuánto cuesta encontrar por un campo y leer un registro de un archivo?

1. Si el archivo está ordenado



2. Si el archivo está ordenado por ese campo

3. Si el archivo tiene una organización dispersa y la fórmula utiliza ese campo

4. Si el archivo está ordenado por otro campo, o el campo no está en la fórmula

Archivo de racimo sobre el campo k

Cubeta 0	3	Aldomar	22	Valencia
	6	García	22	Valencia
	9	Gomez	23	Barcelona
	12	Martínez	22	Madrid
	15	Vilchez	23	Madrid
Cubeta 1	1	Aliaga	23	Bilbao
	4	Cardenas	18	Bilbao
	7	Cerdán	22	Bilbao
	10	Nuñez	20	Madrid
	13	Tarraga	21	Barcelona
Cubeta 2	2	Alcocer	25	Barcelona
	5	Gimenez	21	Barcelona
	8	Lopez	20	Valencia
	11	Romero	24	Valencia
	14	Yañez	21	Barcelona



ALUMNOS

K	Apellido	Edad	Ciudad
2	Alcocer	25	Barcelona
3	Aldomar	22	Valencia
1	Aliaga	23	Bilbao
4	Cardenas	18	Bilbao
7	Cerdán	22	Bilbao
6	García	22	Valencia
5	Gimenez	21	Barcelona
9	Gomez	23	Barcelona
8	Lopez	20	Valencia
12	Martínez	22	Madrid
10	Nuñez	20	Madrid
11	Romero	24	Valencia
13	Tarraga	21	Barcelona
15	Vilchez	23	Madrid
14	Yañez	21	Barcelona

Coste =

FUNCIÓN HASH f(k) = valor mod

CADA CUBETA TIENE 2 BLOGS
CADA BLOQUE TIENE 4 REGISTROS

Machine Translated by Google

Parámetros para la Evaluación de Costes de E/S

Métricas de Archivo y Mesa

- TB : Tamaño del blog de la base de datos (bytes).
- Tx : Número total de registros de la tabla.
- Lx : Longitud en bytes de cada registro de la mesa.
- FBx : Factor de bloque de la tabla (Registros por blog) = TB / Lx
- Bx : Número total de bloques de la tabla = Tx / FBx
- BPCx : Bloques por cubeta en organización dispersa.

DISCUSIÓN: ¿Cuánto cuesta encontrar por un campo y leer un registro de un archivo?

1. Si el archivo está ordenado
2. Si el archivo está ordenado por ese campo
3. Si el archivo tiene una organización dispersa y la fórmula utiliza ese campo
4. Si el archivo está ordenado por otro campo, o el campo no está en la fórmula

Machine Translated by Google

Comparativa de Costes de Búsqueda Sin Índice

1. Archivo No Ordenado

Buscar sobre cualquier atributo mediante escaneo completo.

COSTE DE E/S (BLOQUES)

Bx

Media si existe: Bx / 2

2. Archivo Ordenado

Búsqueda binaria sobre el campo de ordenación físico.

COSTE DE E/S (BLOQUES)

log2 Bx

Si no es el campo de orden: Bx

3. Archivo Disperso

Buscar por el campo clave utilizando la función hash.

COSTE DE E/S (BLOQUES)

BPCx

Típicamente 1 blog si no hay overflow.

Machine Translated by Google

Indexación

2. Técnicas de Indexación

- Concepto y tipos de índices: Primario, secundario, denso y disperso.
- Índices de agrupamiento: Clustering vs non-clustering.
- Índices multinivel: Árboles B+ de orden n.
- Evaluación y análisis de costes de E/S: Modelos matemáticos.

Machine Translated by Google

¿Qué es un Índice en Bases de Datos?

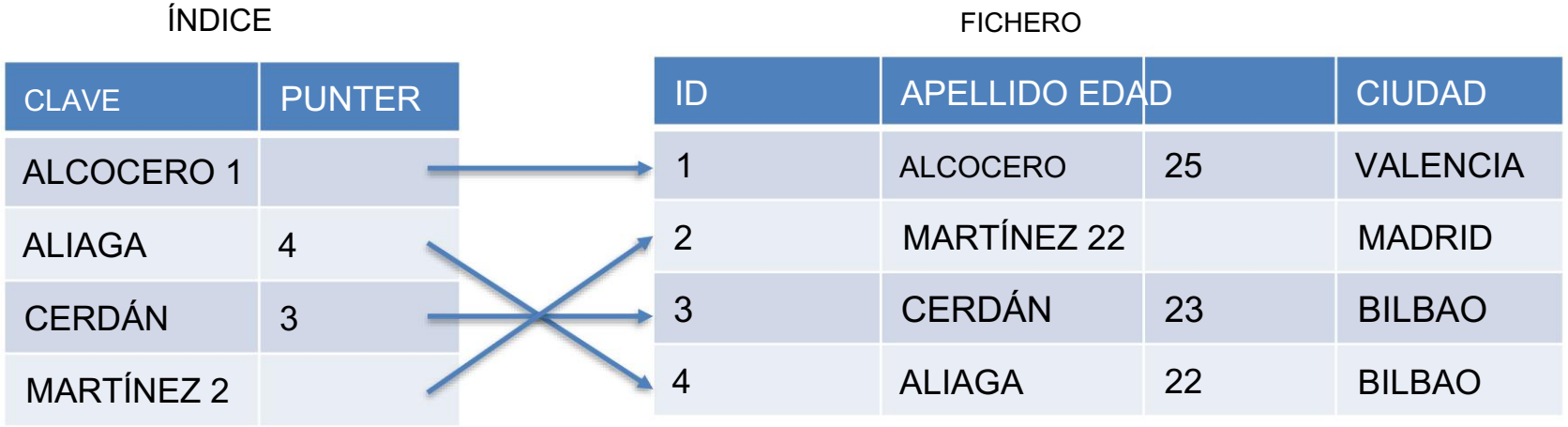
Un índice es una estructura de datos auxiliar diseñada para acelerar el acceso a los registros de un archivo mediante el valor de uno o varios campos de indexación.

Contenido de una Entrada de Índice → (clave, puntero)

- Clave de búsqueda (k): Valor del atributo indexado.
- Puntero (pointer): Dirección física del bloque/sonido registro/s que contiene el dato.

Principio de Rendimiento

El índice es mucho más pequeño en bytes que la mesa original porque sólo aloja la llave y el puntero, permitiendo cargar muchas entradas en un solo bloque de disco.



CLAVE	PUNTER
ALCOCERO 1	
ALIAGA	4
CERDÁN	3
MARTÍNEZ 2	

ID	APELLIDO	EDAD	CIUDAD
1	ALCOCERO	25	VALENCIA
2	MARTÍNEZ 22		MADRID
3	CERDÁN	23	BILBAO
4	ALIAGA	22	BILBAO

Machine Translated by Google

Clasificación Completa de los Índices

Primario vs Secundario

Primario: Definido sobre los atributos de la clave primaria.

Secundario: Definido sobre otros atributos que NO son la clave primaria.

Agrupamiento (Clustering)

Agrupamiento: Coincide con el orden físico de los registros a disco (máximo 1 por mesa).
Archivo está ordenado

No agrupamiento: El orden del índice es diferente del orden del archivo.

Denso vs Disperso

Denso: Existe una entrada en el índice para cada registro.

Disperso: Sólo hay entrada para algunos registros (uno por cada bloque de datos).

Único vs No único

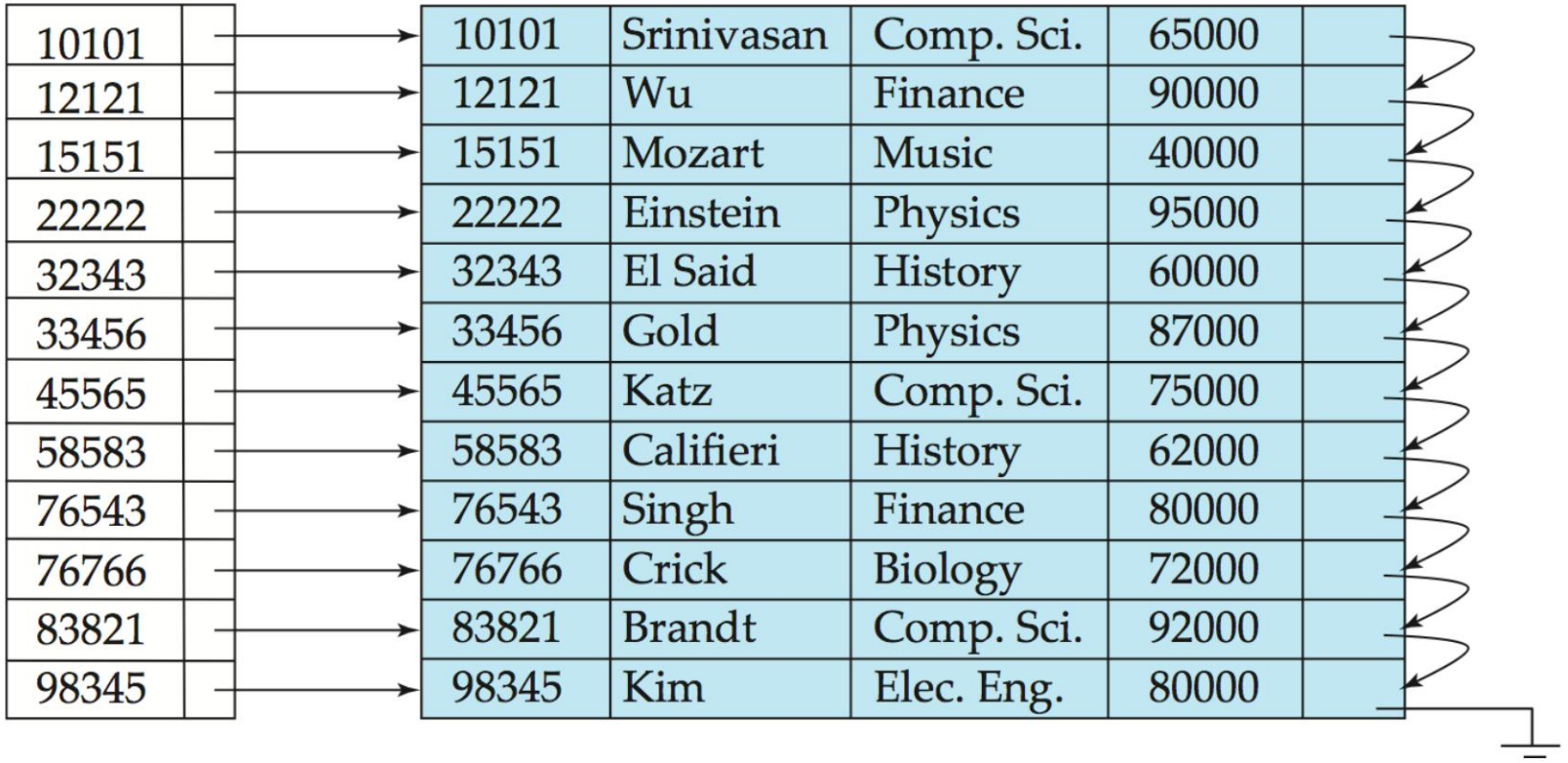
ÚNICO: Sobre atributos con restricción de UNICIDAD (UNIQUE)

No ÚNICO: Sobre atributos SIN restricción de UNICIDAD

Machine Translated by Google

Clasificación Completa de los Índices (Ejemplos)

ÍNDICE DENSO DE AGRUPAMENTO Y PRIMARIO



10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

PRIMARIO → LA CLAVE DEL ÍNDICE ES CLAVE PRIMARIO EN EL ARCHIVO

AGRUPAMENTO → ESTÁ ORDENADO EL ARCHIVO POR EL CAMPO QUE ES CLAVE DEL ÍNDICE

DENSO → CADA ENTRADA DE ÍNDICE ENLAZA CON UN REGISTRO

COST D'E/S DE LA CERCA

$[\log_2 Bl_{cx}] + 1$

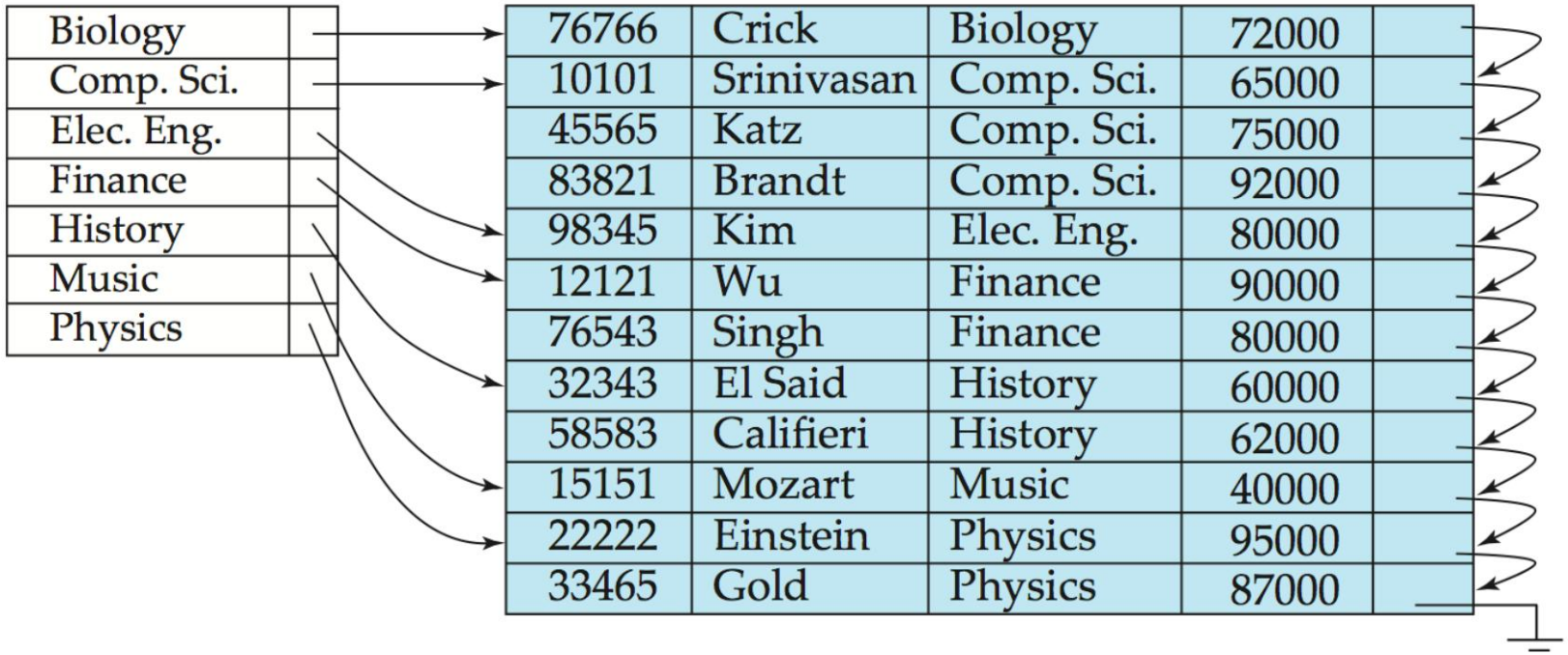
Cerca binària sobre els blocs d'índex (...+1 lectura del bloc de dades final.

Bl_{cx}

Machine Translated by Google

Clasificación Completa de los Índices (Ejemplos)

ÍNDICE DISPEROS DE AGRUPAMENTO SECUNDARIO Y NO ÚNICO



La cerca localitza el bloc candidat i després s'explora el bloc en memòria.

COST D'E/S DE LA CERCA

$\lceil \log_2 B|cx| \rceil + 1$

SECUNDARIO → LA CLAVE DEL ÍNDICE NO ES CLAVE PRIMARIO EN EL ARCHIVO

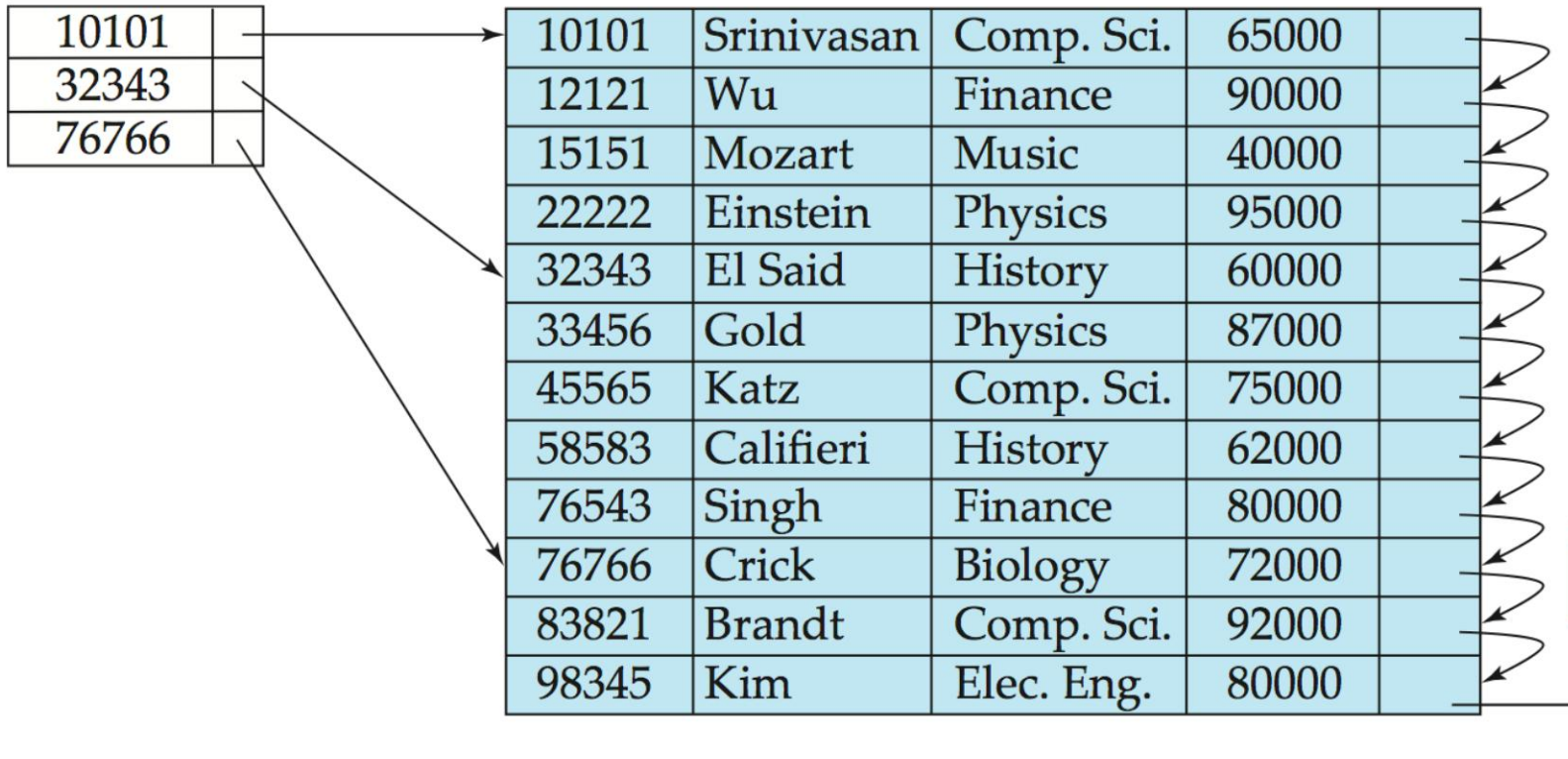
AGRUPAMENTO → ESTÁ ORDENADO EL ARCHIVO POR EL CAMPO QUE ES CLAVE DEL ÍNDICE

DISPEROS → CADA ENTRADA DE ÍNDICE ENLAZA CON EL PRIMER REGISTRO DEL GRUPO QUE TIENEN EL MISMO VALOR EN EL CAMPO QUE ES CLAVE DEL ÍNDICE

Machine Translated by Google

Clasificación Completa de los Índices (Ejemplos)

ÍNDICE DISPEROS DE AGRUPAMENTO PRIMARIO



10101	
32343	
76766	

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

La cerca localitza el bloc candidat i després s'explora el bloc en memòria.

COST D'E/S DE LA CERCA

$\lceil \log_2 B_{lcx} \rceil + 1$

PRIMARIO → LA CLAVE DEL ÍNDICE ES CLAVE PRIMARIO EN EL ARCHIVO

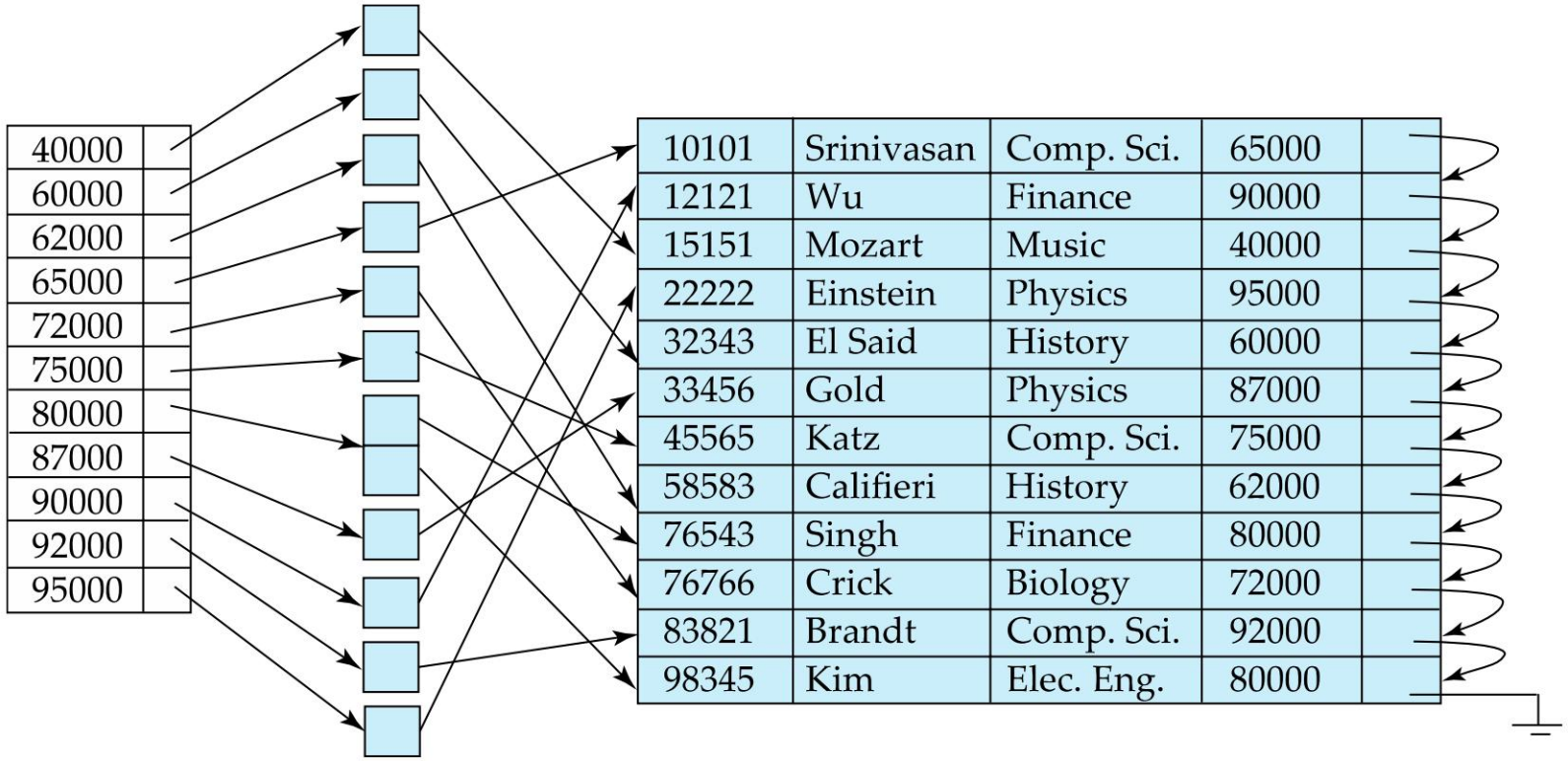
AGRUPAMENTO → ESTÁ ORDENADO EL ARCHIVO POR EL CAMPO QUE ES CLAVE DEL ÍNDICE

DISPEROS → CADA ENTRADA DE ÍNDICE ENLAZA CON EL PRIMER REGISTRO DE UN GRUPO ORDENADO POR EL CAMO DEL ARCHIVO QUE SE CLAVE EN EL ÍNDICE

Machine Translated by Google

Clasificación Completa de los Índices (Ejemplos)

ÍNDICE DENSO, NO AGRUPAMIENTO SECUNDARIO Y NO ÚNICO (SI ES DE NO AGRUPAMIENTO TÉ QUE SER DENSO)



SECUNDARIO → LA CLAVE DEL ÍNDICE NO ES CLAVE PRIMARIO EN EL ARCHIVO

NO AGRUPAMENTO → NO ESTÁ ORDENADO EL ARCHIVO POR EL CAMPO QUE ES CLAVE EN EL ÍNDICE

DENSO → CADA ENTRADA DE ÍNDICE ENLAZA CON UN REGISTRO

FÓRMULA DE COST TOTAL D'E/S

$$\lceil \log_2 BI_{cx} \rceil + BIS_{cvx} + C_x$$

Cerca binària en índex + lectura de blocs de punters + lectura dels blocs de dades ().

Machine Translated by Google

Actualización de Índices en Inserciones y Borrados

En caso de Inserción

Índices Densos: Hay que añadir obligatoriamente una nueva entrada en el índice (salvo que la clave ya exista en índices no únicos).

Índices Dispersos: No es necesario añadir ninguna entrada nueva a menos que la inserción cree físicamente un nuevo bloque de datos.

En caso de Borrado

Índices Densos: Se elimina siempre la entrada correspondiente a la tupla borrada.

Índices Dispersos: Sólo se actualiza la entrada si la clave eliminada era precisamente la clave guía que figuraba en el índice.

Machine Translated by Google

La Necesidad de Índices Multinivel

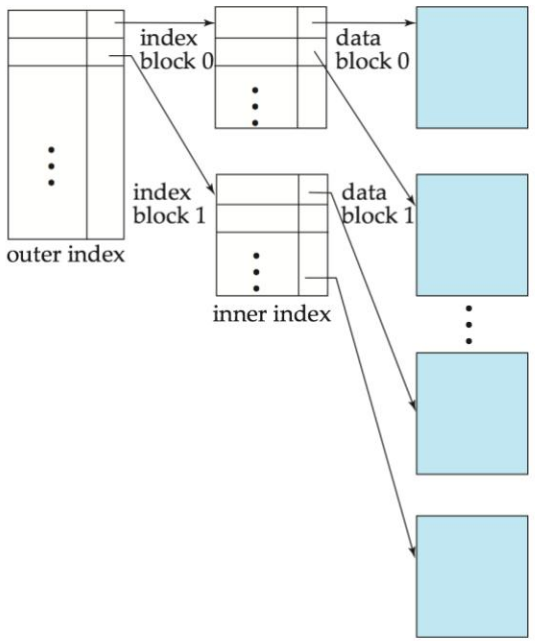
Si un archivo de índice mononivel es tan grande que ocupa miles de bloques, la búsqueda binaria sobre el propio índice comienza a requerir demasiadas lecturas de E/S a disco.

Solución: Índice sobre el Índice

Se crea un índice externo (outer index) que indexa los bloques del índice interno (inner index). El proceso se repite jerárquicamente hasta que la raíz cabe en un único bloque de disco.

Estructuras de Árbol en SGBD

Para evitar tener que reorganizar bloques secuenciales cuando se producen inserciones y divisiones, los sistemas de bases de datos utilizan estructuras dinámicas equilibradas: Árboles B+.



Machine Translated by Google

Árbol B+ de Orden n: Propiedades Estructurales

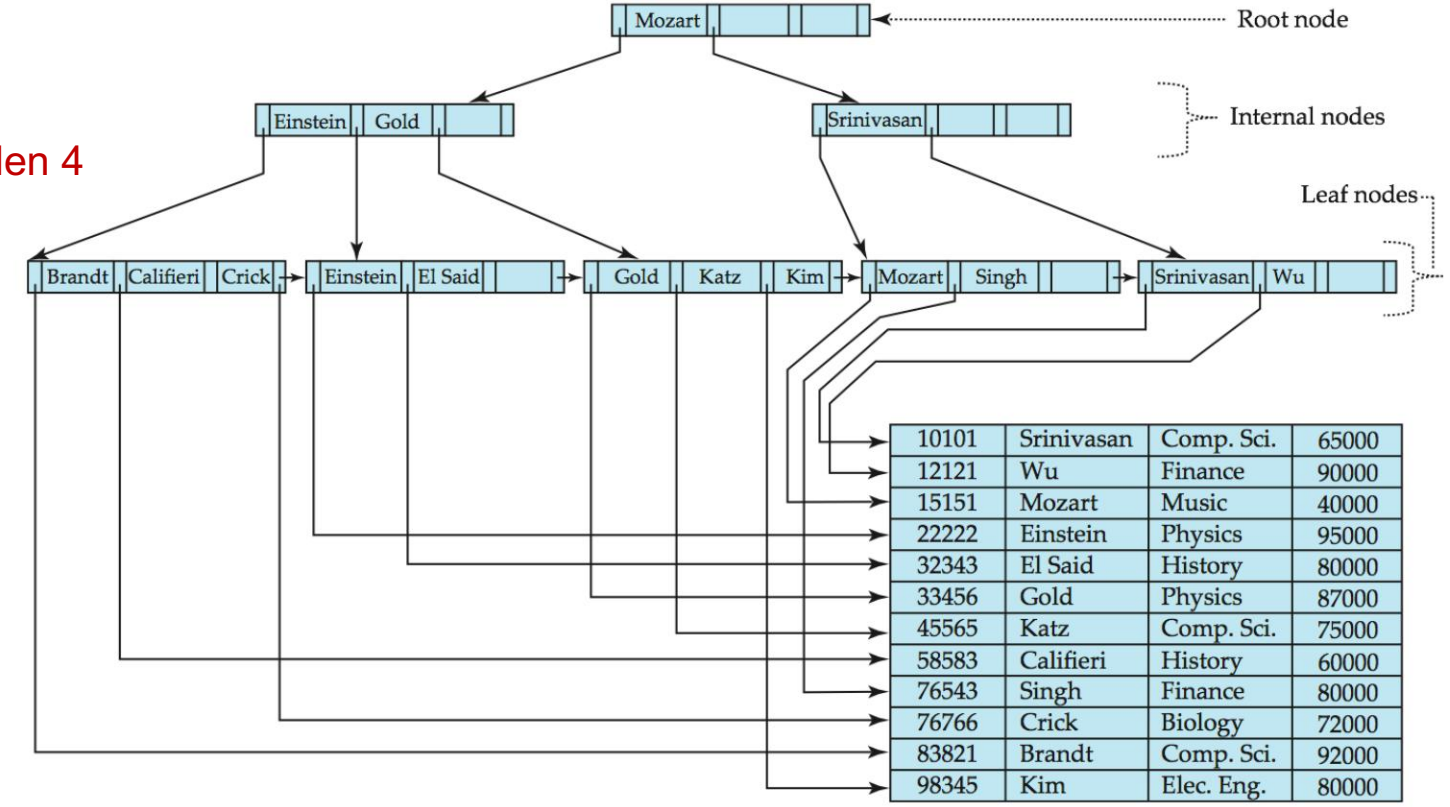
Nodos Interiores

- Tienen como máximo n punteros en nodos hijos.
- Excepto la raíz, tienen al menos n / 2 punteros.
- Un nodo con k punteros a hijos aloja exactamente k - 1 claves ordenadas.

Nodos Hoja

- Tienen como máximo n - 1 punteros en registros de la tabla.
- Tienen al menos (n - 1) / 2 punteros a registros.
- Contienen un puntero adicional en el siguiente nodo hoja.

Ejemplo árbol B+ de orden 4



10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	80000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	60000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Machine Translated by Google

Capacidad y Cálculo de la Altura en Árboles B+

Capacidad de Registros

Un árbol B+ de orden n y altura k permite alojar un número máximo de claves dado por la fórmula:

CAPACIDAD MÁXIMA (M)

$$m = n^{k-1} \cdot (n - 1)$$

Cálculo de la Altura (k)

Para indexar m claves en un árbol B+ de orden n, el número de niveles requerido es:

ALTURA DEL ÁRBOL (K)

$$k = \log_n m$$

Operación de Búsqueda en Árboles B+

Algoritmo de Descenso

1. Se comienza por la rarel (residente habitualmente en memoria RAM).
2. En cada nodo interior, se compara la clave buscada con las claves del nodo para elegir el puntero hijo adecuado.
3. Se desciende de nivel en nivel hasta llegar al nodo hoja correspondiente.
4. En la hoja, se lee el puntero y se accede al bloque del archivo de datos.

Coste de E/S de la Búsqueda Puntual

COSTE TOTAL (BLOCES DE DISCO)

$$H_{Icx} + 1$$

H_{Icx}

Donde esta es la altura del árbol B+ (típicamente 2, 3 o 4 accesos) más 1 lectura final del bloque de datos.

Machine Translated by Google

Árboles B+: La Gran Estructura de Indexación

EQUILIBRIO PERFECTO

Altura Uniforme

Todos los nodos hoja están exactamente en el mismo nivel de profundidad; cualquier búsqueda tiene lo mismo coste.

AUTO-REORGANIZACIÓN

Cambios Locales

Las inserciones y borrados se ajustan mediante divisiones (splits) o fusiones (mérgenes) sin rehacer todo el árbol.

RECORRIDO RÁPIDO

Hojas Enlazadas

Los nodos hoja están enlazados en una lista secuencial para resolver consultas por rango de forma óptima.

Machine Translated by Google

Inserción en Árboles B+: Algorismo de Insercio y Mecanismo de División

Algoritmo de Inserción: 1.
Se localiza la hoja correcta.

2. Insertar el nuevo registro

3. Si clave no está presente en la hoja:

    1. Si tiene espacio libre (< n-1 claves), se inserta ordenadamente.
    2. Si está llena, se divide el nodo (split).

1. División de Nodo Hoja (Leaf Split)

El contenido se reparte entre la hoja existente y una nueva creada. La llave menor de la segunda hoja se copia y sube hacia el nodo padre.

2. División de Nodo Interior (Internal Split)

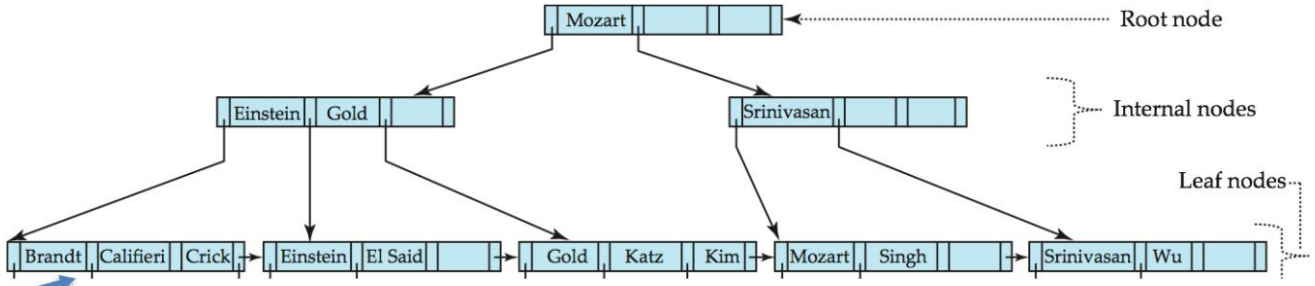
Si el padre también está lleno, se divide en dos nodos y la clave central se promociona en el nivel superior. Si es necesario dividir la raíz, se crea una raíz nueva y la altura crece en 1.

Machine Translated by Google

Inserción en Árboles B+: Algorismo de Insercio y Mecanismo de División

1. División de Nodo Hoja (Leaf Split)

El contenido se reparte entre la hoja existente y una nueva creada. La llave menor de la segunda hoja se copia y sube hacia el nodo padre.



Insertar registro “Adams”

Encontrar la hoja.

El nodo hoja está lleno.

Si insertaremos "Adams", tendría

4 claves en orden:["Adams", "Brandt", "Califiere", "Crick"].

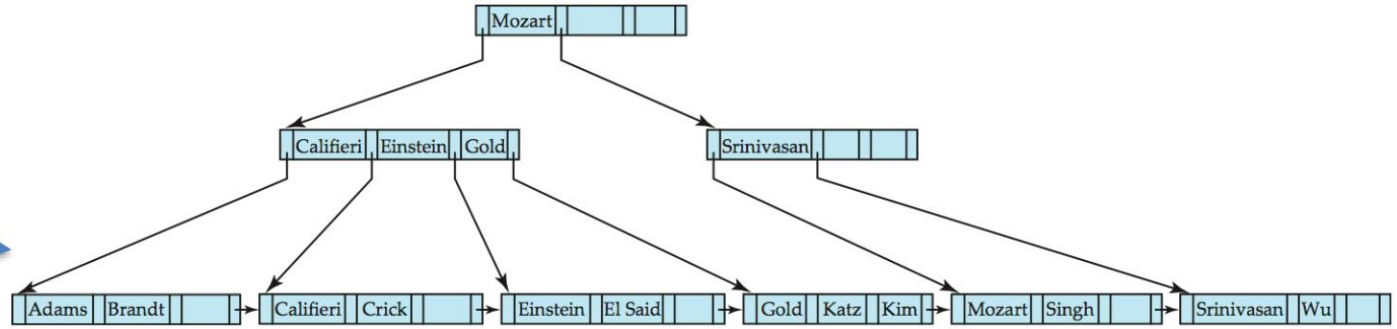
Se divide en 2 hojas de dos elementos:

•Hoja Izquierda: [Adams | Brandt]

•Hoja Derecha: [Califerio | Crick]

•La clave más pequeña del nuevo nodo ["Califieri“]

se copia y sube al nodo padre.



Machine Translated by Google

Inserción en Árboles B+: Algorismo de Insercio y Mecanismo de División

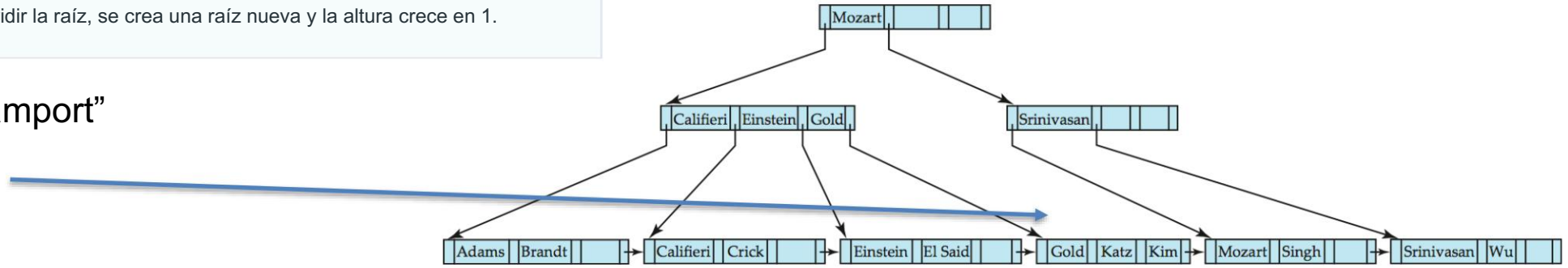
2. División de Nodo Interior (Internal Split)

Si el padre también está lleno, se divide en dos nodos y la clave central se promociona en el nivel superior. Si es necesario dividir la raíz, se crea una raíz nueva y la altura crece en 1.

Insertar registro “Lamport”

Encontrar la hoja.

El nodo hoja está lleno.
Si insertaremos "Adams",
tendría 4 calus en orden:["Gold", "Katz", "Kim", "Lamport"].



Se divide en 2 hojas de dos elementos:

•Hoja Izquierda: [Gold | Kaz]
•Hoja Derecha: [Kim | Lamport]

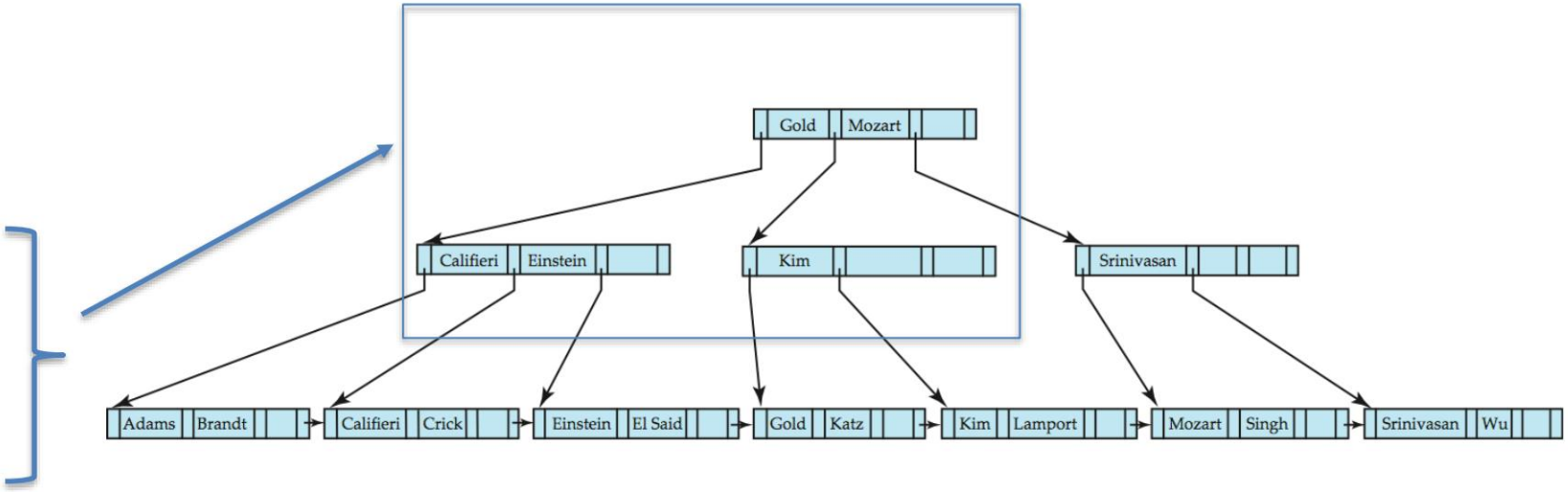
•La clave más pequeña del nuevo nodo

[“Kim“] se copia y sube al nodo padre.

El padre, al recibir Kim, pasa a trindre 4 llaves
[“Califerio”, “Einstein”, “Gold”, “Kim”]

División de nodo interior:

- Llave del medio "Gold" sube al superior (raíz)
- El nodo izquierdo conserva [“Califerio”, “Einstein”]
- El nuevo nodo derecho conserva el mayor “Kim”



Machine Translated by Google

Borrado en Árboles B+: Fusión y Redistribución

Algoritmo de Borrado: Se elimina la llave y el puntero de la hoja. Si el nodo queda por debajo del mínimo permitido (subdesbordamiento/underflow), es necesario reequilibrar.

Opción A: Redistribución con el Hermano

Si el nodo hermano adyacente tiene más claves del mínimo, se toma en préstamo una clave y se actualiza la clave separadora en el nodo padre.

Opción B: Fusión de Nodos (Merge)

Si el hermano tampoco tiene llaves de sobra, se fusionan los dos nodos en uno solo y se elimina la llave separadora del padre (pudiendo propagar la fusión hacia arriba).

Machine Translated by Google

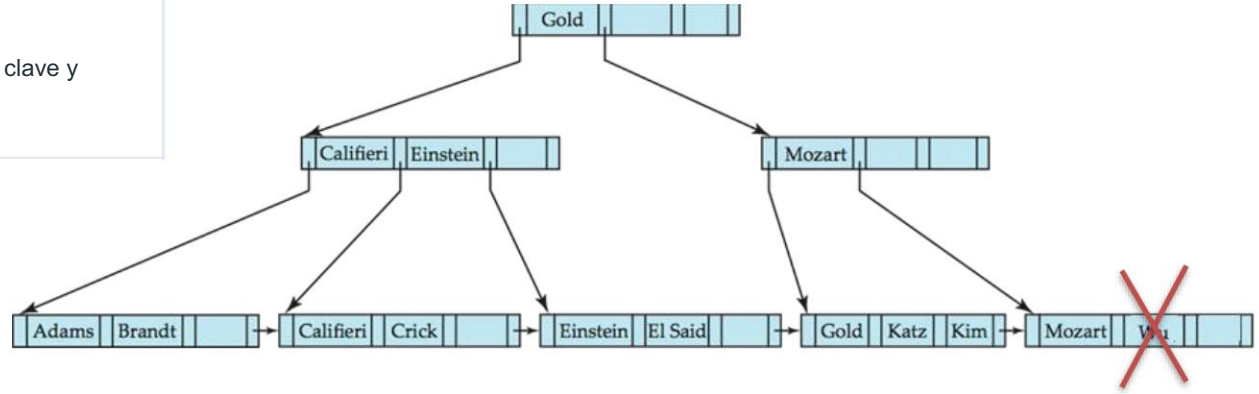
Borrado en Árboles B+: Fusión y Redistribución

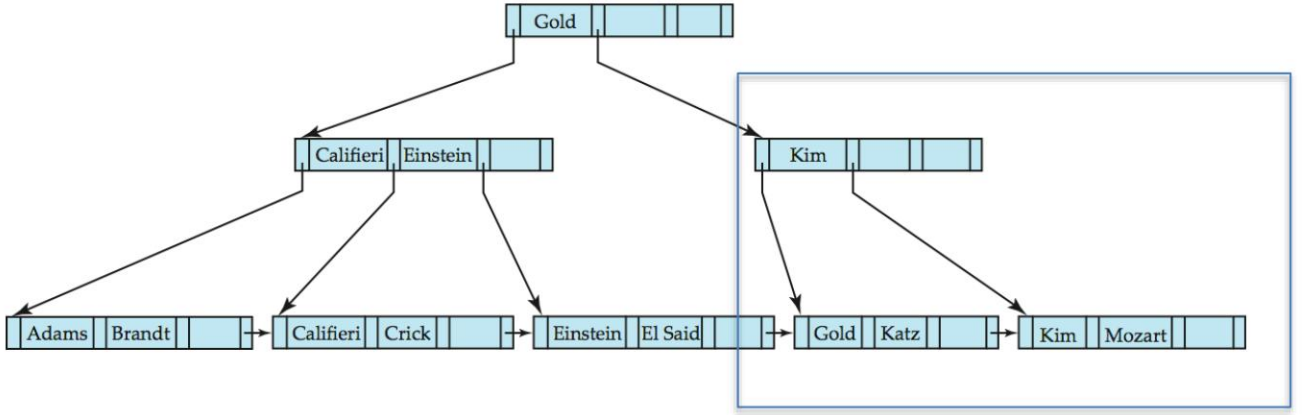
Algoritmo de Borrado: Se elimina la llave y el puntero de la hoja. Si el nodo queda por debajo del mínimo permitido (subdesbordamiento/underflow), es necesario reequilibrar.

Opción A: Redistribución con el Hermano

Si el nodo hermano adyacente tiene menos claves del máximo, se toma en préstamo una clave y se actualiza la clave separadora en el nodo padre.

Eliminar registro “Whu”





Machine Translated by Google

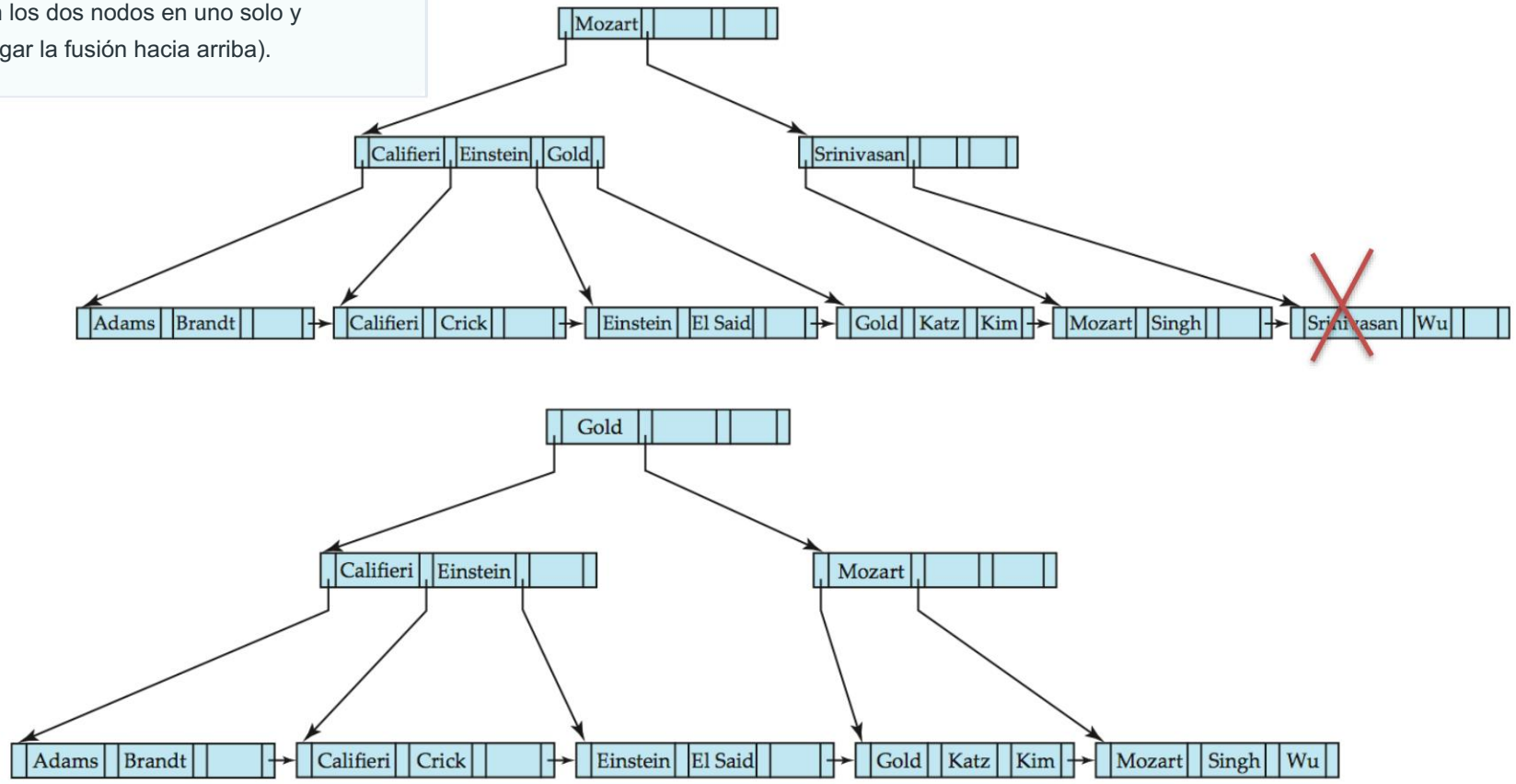
Borrado en Árboles B+: Fusión y Redistribución

Algoritmo de Borrado: Se elimina la llave y el puntero de la hoja. Si el nodo queda por debajo del mínimo permitido (subdesbordamiento/underflow), es necesario reequilibrar.

Opción B: Fusión de Nodos (Merge)

Si el hermano tampoco tiene llaves de sobra, se fusionan los dos nodos en uno solo y se elimina la llave separadora del padre (pudiendo propagar la fusión hacia arriba).

Eliminar registro “Sriniwasan”



Machine Translated by Google

Tabla Resumen de Costes de E/S en SGBD

Estructura de Almacenamiento	Búsqueda Puntual por Clave	Búsqueda por Rango
Archivo No Ordenado (Heap)	Bx	Bx
Archivo Ordenado	log2 Bx	log2 Bx + blocs_rango
Archivo Disperso (Hash)	BPCx	Bx (ineficiente)
Índice Primario Denso Ordenado	log2 BIcx + 1	log2 BIcx + blocs_dato
Índice B+ Primario / Único	HIcx + 1	HIcx + hojas_rango + blocs_dato
Índice B+ Secundario No Único	HIcx + BIScvx + Cx	HIcx + listas_punteros + Cx

Machine Translated by Google

Ejercicios de Discusión y Deducción

Cuestiones sobre Organización Físico

- ¿En qué escenarios reales de aplicación es preferible utilizar una organización de tipo heap?
- ¿Qué ventajas tiene el direccionamiento cerrado frente al direccionamiento abierto cuando el factor de carga aumenta?
- ¿Por qué un archivo ordenado puede tener como máximo un único índice de agrupamiento (clustering)?

Cuestiones sobre Árboles B+

- Si tenemos un árbol B+ de orden 4, ¿cuál es el número mínimo y máximo de claves en una hoja?
- ¿Qué ocurre cuando una hoja sufre un underflow y sus hermanos no tienen claves para prestar?
- ¿Por qué los nodos de un árbol B+ se dimensionan exactamente para coincidir con el tamaño de página de la BD?

Machine Translated by Google

UNIVERSIDAD DE VALENCIA • ETSE-UV

Tema 2.b: Organización e Indexación

Sistemas de Gestión de Bases de Datos • Profesor: Javier Sevilla

---

Lecturas recomendadas: Silberschatz, Korth & Sudarshan (Cap. 11: Indexing and Hashing)

Departamento de Informática — Universidad de Valencia